

PROGRAMAREA CALCULATOARELOR ȘI LIMBAJE DE PROGRAMARE

Tema #2 Alocare dinamică de memorie. Tablouri. Șiruri de caractere.

Deadline soft: 03.12.2025 23:55. Deadline hard: 6.12.2025 23:55

Coordonator: Dorinel Filip

Responsabili temă: Mihai-Cătălin Stan, Raluca-Monica Bîrlădeanu, Matei-Teodor Bîrleanu

Changelog:

- 17.11.2025: Publicare tema 2.
Vă rugăm să adresați toate întrebările legate de această temă pe forumul asociat temei 2 de pe site-ul de curs. Orice clarificare / corectare / actualizare legată de enunț, checker sau lucruri administrative etc, vor fi anunțate exclusiv pe forumul dedicat temei 2.
- 24.11.2025: Publicare checker. A fost adăugată o observație adițională pentru extindere problemei, pe care o puteți găsi [aici](#). Au fost adăugate exemple pentru task-urile 2 și 3.

Contents

Definirea problemei - Battleships	3
Extinderea problemei - Statistici, Criptare, Decriptare	7
Algoritmul Hill Cipher	11
Regulament	15
Arhivă	15
Checker	15
Punctaj	15
Reguli și precizări	15
Alte precizări	16

Definirea problemei - Battleships

Enunț

Gigel și Mihai sunt studenți în anul I la Facultatea de Automatică și Calculatoare. Într-o seară petrecută în regie, aceștia lucrau la analiză matematică, amintindu-și cu nostalgie de perioada copilăriei, când grijile erau mai puține și timpul liber mai generos.

Aceștia și-au reamintit de jocul Battleships, pe care au decis să îl joace pe hârtie în mica lor pauză. S-au pus de comun acord să joace pe o tablă făcută de ei cu N linii și M coloane, cu nave de lungimi de la 1 până la 5 pătrățele. Pentru a determina numărul de nave, aceștia au creat următoarele formule bazate pe dimensiunea tablei:

Nume	Identificator	Lungime	Nr_Nave
Shinano	S	5	$\frac{N*M}{70}$
Yamato	Y	4	$\frac{N*M}{55}$
Belfast	B	3	$\frac{N*M}{40}$
Laffey	L	2	$\frac{N*M}{30}$
Albacore	A	1	$\frac{N*M}{20}$

După ce au plasat navele pe tabla de joc, au ales pe rând câte o poziție pe tablă pe care să o atace până când unul dintre ei a rămas fără unități. Câștigătorul unui joc se definește ca fiind jucătorul care distruge toate navele inamice înainte ca navele sale să fie distruse. O navă este distrusă atunci când toate punctele navei au fost lovite sau când punctul în care a fost plasată nava a fost lovit. Spre exemplu, dacă plasăm o navă de tip Laffey orizontal pe poziția (1, 1), aceasta va ocupa căsuțele (1, 1) și (1, 2). Pentru a o distruge vă trebuie fie să lovim coordonata (1, 1) la care a fost plasată, fie în ordine (1, 2) și (1, 1).

Scrieți un program în C care să primească ca input informații despre joc și să simuleze jocul lui Gigel și Mihai pe care îl aveți descris mai sus.

Restricții și precizări

- Nu există limitări privind numărul de meciuri, numărul de nave sau dimensiunile tablelor de joc.
- Toate tablourile necesare vor fi **alocate dinamic**. Nu alocați resursele static și dealocați resursele de care nu mai aveți nevoie. Dacă pe un test sunt detectate memory leak-uri nu veți primi punctajul aferent pentru acesta.
- Atenție! Memoria de pe platforma VMChecker este limitată pentru această problemă. Încercați să folosiți eficient resursele alocate pentru un punctaj complet.

Date de intrare

Toate datele se vor citi de la **STDIN**.

- Pe prima linie se află un număr întreg **J** care reprezintă numărul de meciuri pe care cei doi au de gând să le joace.
- Pe următoarea linie se găsesc două numere întregi **N** și **M** care reprezintă numărul de linii respectiv coloane a primului meci. Indexarea tablei se va face din colțul din stânga sus și va începe de la coordonatele (1, 1).
- Fiecare jucător are propria lui tablă de joc pe care își amplasează navele.
- Pe următoarele K linii, perechi de forma **Tip Orientare Coordonata_X Coordonata_Y**, unde Tip este inițiala numelui navei pe care o plasează. Orientare va fi un caracter 'H' - pentru orizontal - sau 'V' - pentru vertical. X și Y vor fi numere întregi care reprezintă coordonatele de start ale navei. Navele plasate orizontal vor avea coordonata de start capătul din stânga, iar cele plasate vertical vor avea coordonata de start capătul de jos.
- Pe următoarele linii, perechi de forma **Atac_X Atac_Y** care reprezintă coordonatele la care jucătorul curent vrea să atace. Jucătorii atacă alternativ.
- Pentru $J > 1$, cele trei intrări menționate mai sus se vor repeta pentru restul de $J - 1$ meciuri.
- Atacurile se citesc până când unul dintre jucători pierde.
- Dacă, în faza de pregătire a jocului, un jucător plasează o navă invalid, acesta trebuie să o plaseze din nou.
- Dacă un jucător atacă o casuță deja atacată sau o navă deja distrusă, își pierde tura, iar atacul este înregistrat ca lovitură ratată în statistică.

Între oricare doi tokeni vizibili din input există exact un spațiu. La finalul oricărei linii din input există un caracter **newline**.

```

1 J
2 N M
3 Tip_0 Orientare_0 X_0 Y_0
4 Tip_1 Orientare_1 X_1 Y_1
5 Tip_2 Orientare_2 X_2 Y_2
6 ...
7 Tip_{k - 1} Orientare_{k - 1} X_{k - 1} Y_{k - 1}
8 Atac_X_0 Atac_Y_0
9 Atac_X_1 Atac_Y_1
10 ...

```

Date de ieșire

Toate datele se vor afișa la **STDOUT**.

- După amplasarea celor K nave, se vor afișa cele două table. Între fiecare două elemente se va lăsa un spațiu, iar între fiecare două linii un caracter **newline**. Între cele două matrici se vor lăsa două caractere **newline**.
- La amplasarea navelor, respectiv în timpul atacurilor, se vor afișa dacă este cazul mesaje de eroare.
- După fiecare atac se va afișa un singur mesaj corespunzător precum "Jucătorul 1 a lovit o navă Laffey la coordonata (1, 2)".
- La finalul fiecărei runde, se va afișa câștigătorul runde respective.

Mesaje output posibile

Mesaj	Caz de afișare
Jucătorul A a lovit o navă B la coordonatele (X, Y) .	La lovirea cu succes a unei nave
Jucătorul A a distrus o navă B plasată între coordonatele (X_{start}, Y_{start}) și (X_{end}, Y_{end}) .	La distrugerea unei nave
Jucătorul A a ratat o lovitură la coordonatele (X, Y) .	La ratarea unei lovituri
Coordonatele (X, Y) au fost deja atacate de jucătorul A .	La lovirea repetată a unor coordonate
Nava P nu poate fi amplasată <i>vertical/orizontal</i> la coordonatele (X, Y) .	La amplasarea incorectă a unei nave
Jucătorul A a câștigat.	La câștigarea jocului

Exemplul 1**Date de intrare**

```

1 1
2 6 6
3 A H 1 1
4 L V 1 1
5 L H 1 1
6 L V 3 4
7 A V 2 2
8 1 1
9 2 2
10 1 2
11 2 4
12 2 2

```

Date de ieșire

```

1 Nava Laffey nu poate fi amplasata vertical la coordonatele (1, 1).
2 1 0 0 0 0 0
3 0 0 0 2 0 0
4 0 0 0 2 0 0
5 0 0 0 0 0 0
6 0 0 0 0 0 0
7 0 0 0 0 0 0
8
9 2 2 0 0 0 0
10 0 1 0 0 0 0
11 0 0 0 0 0 0
12 0 0 0 0 0 0
13 0 0 0 0 0 0
14 0 0 0 0 0 0
15 Jucatorul 1 a distrus o nava Laffey plasata intre coordonatele (1, 1) si (1, 2).
16 Jucatorul 2 a ratat o lovitura la coordonatele (2, 2).
17 Jucatorul 1 a ratat o lovitura la coordonatele (1, 2).
18 Jucatorul 2 a lovit o nava Laffey la coordonatele (2, 4).
19 Jucatorul 1 a distrus o nava Albacore plasata intre coordonatele (2, 2) si (2, 2).
20 Jucatorul 1 a castigat.

```

Explicație 1

1. Inițial se menționează numărul de jocuri, în cazul acestui exemplu un singur joc are loc.
2. Ulterior, se introduc lungimea și înălțimea tablei de joc, în cazul de față tabla va avea 6 linii și 6 coloane.
3. Primul jucător își așează prima navă, de dimensiune 1, în poziție orizontală pe coordonatele (1, 1).
4. Al doilea jucător își plasează prima sa navă, de dimensiune 2, în poziție verticală la coordonatele (1, 1). Un mesaj de eroare este afișat întrucât o navă de lungime 2 plasată vertical la coordonatele (1, 1) ar ieși din tabla de joc.
5. Jucătorul al doilea reîncearcă apoi amplasarea navei în poziție orizontală la coordonatele (1, 1). De data aceasta nava este plasată, capul navei se află pe poziția (1, 1) și se continuă în dreapta pe coordonata (1, 2).
6. Plasarea navelor continuă până când ($\frac{6 \cdot 6}{20} =$) 1 navă de dimensiunea 1 și ($\frac{6 \cdot 6}{30} =$) 1 navă de dimensiunea 2 sunt amplasate. (Conform cu liniile 6-7 din datele de intrare)
7. Se afișează configurația tablelor, întâi cea a primului jucător, iar apoi cea pentru al doilea jucător.
8. Primul jucător atacă coordonatele (1, 1), lovind capul navei de lungime 2 a celui de al doilea jucător. Deoarece a lovit capul acestei nave, este distrusă fără a fi necesară lovirea celorlalte coordonate pe care se află. De asemenea, este afișat un mesaj corespunzător distrugerii navei - "Jucatorul 1 a distrus o nava Laffey plasată între coordonatele (1, 1) și (1, 2)." - unde (1, 1) sunt coordonatele de start ale navei, iar (1, 2) sunt coordonatele capătului navei. Atunci când distrugem o navă, afișăm doar mesajul de distrugere a acesteia, nu și pe cel de lovire cu succes a navei.
9. Ulterior, al doilea jucător atacă coordonatele (2, 2), nicio navă a jucătorului 1 nu este lovită. Un mesaj în care se anunță că lovitura a fost ratată este afișat.
10. Primul jucător atacă coordonatele (1, 2), deoarece nava de la acea poziție a fost deja distrusă, lovitura este considerată ca fiind una ratată și un mesaj corespunzător este afișat.
11. Al doilea jucător atacă coordonatele (1, 1) și distruge o navă de tip Albacore a primului jucător, încă o dată este afișat un mesaj pentru anunțarea distrugerii navei.
12. Ultimul atac este al primului jucător, care atacând coordonatele (2, 2) distruge ultima navă a celui de al doilea jucător. Este afișat atât un mesaj corespunzător distrugerii navei, cât și un mesaj de final de joc.

Extinderea problemei - Statistici, Criptare, Decriptare

Enunț

Pentru că Gigel și Mihai sunt foarte competitivi, după ce s-au săturat de jucat partidele de Battleships, s-au decis să calculeze niște statistici pentru a vedea cine este jucătorul mai bun. Prin urmare, aceștia și-au calculat acuratețea în fiecare meci, definită prin formula:

$$acc_meci = \frac{nr_lovituri_nimerite}{nr_lovituri_totale}$$

Pe deasupra, au calculat acuratețea totală ca fiind media ponderată a acurateților meciurilor în funcție de numărul de nave din fiecare meci (notat K_i):

$$acc_total = \frac{\sum_{i=1}^J (acc_meci_i * K_i)}{\sum_{i=1}^J K_i}$$

După ce și-au calculat fiecare acuratețile, vor să vadă cât de multe greșeli își mai permitau sau cât de bine trebuiau să joace în anumite meciuri pentru a atinge diverse obiective. Aceștia și-au setat 4 cerințe:

1. Numărul minim de meciuri în care ar fi trebuit să aibă 100% acuratețe pentru a avea o acuratețe totală mai mare sau egală cu X%.
2. Numărul minim de lovituri pe care nu ar fi trebuit să le rateze într-un meci A pentru a avea acuratețea în meciul A mai mare sau egală cu X% .
3. Numărul maxim de meciuri în care ar fi putut avea 0% acuratețe pentru a avea o acuratețe totală mai mare sau egală cu X%.
4. Numărul maxim de lovituri pe care le-ar fi putut rata într-un meci A pentru a avea acuratețea în meciul A mai mare sau egală cu X%.

Pentru a își păstra statisticile private, Gigel și Mihai au decis să își creeze atât cerințele cât și statisticile cu o cheie. Detaliile de implementare a criptării și decriptării vor fi explicate după secțiunea **Date de ieșire**.

Ajutați-i pe Gigel și pe Mihai să scrie un cod în C care să calculeze și să creeze aceste statistici, și să decripteze cerințele.

Date de intrare

- Pe prima linie se va citi cheia sau caracterul "-". În cazul citirii caracterului "-", comenzile vor fi primite necriptate, iar statisticile cerute vor fi afișate în clar.
- Pe următoarele linii se vor citi câte un mesaj pe fiecare linie, mesajele vor fi codificate astfel:
Player|Tip1|AAA|BBB.BB|Tip2. Dintre acestea:
 - Player - Un caracter, poate fi 'O' sau 'T' reprezentând faptul că se cere a fi afișate statisticile pentru jucătorul 1, respectiv 2.
 - Tip1 - Un caracter, poate fi 'U' sau 'T' reprezentând numărul de meciuri pentru care dorim statisticile. Pentru un meci, respectiv pentru tot jocul.
 - AAA - Un întreg de 3 cifre reprezentând indicele meciului pentru care dorim statisticile.
 - BBB.BB - Un număr de 3 cifre, urmat de un caracter "." și de 2 zecimale reprezentând acuratețea la care dorim să ne raportăm statisticile
 - Tip2 - Un șir de 3 caractere, poate fi "MIN" sau "MAX". Pentru "MIN" veți determina numărul minim de meciuri cu 100% acuratețe ori lovituri ratate în minus pentru a ajunge la o acuratețe peste X%. Pentru "MAX" veți determina numărul maxim de meciuri cu 0% acuratețe ori numărul maxim de lovituri ratate în plus pentru a păstra o acuratețe peste X%.
- Pe ultima linie se va citi un caracter 'Q'.

Toate datele se vor citi de la **STDIN**, în continuarea datelor de la primul task.

```
1 Key
2 Command_1
3 Command_2
4 ...
5 Q
```

Date de ieșire

Toate statisticile cerute se vor afișa la **STDOUT**.

- Pentru cerințele 1 și 3 vom afișa acuratețea totală sub forma unui număr de 3 cifre și 2 zecimale separate prin caracterul ".". Apoi, separat printr-un alt caracter ".", numărul de meciuri rezultat din rezolvarea cerinței.
- Pentru cerințele 2 și 4 vom afișa acuratețea meciului la care ne referim sub forma unui număr de 3 cifre și 2 zecimale separate prin caracterul ".". Apoi, separat printr-un alt caracter ".", numărul de lovituri rezultat din rezolvarea cerinței. Dacă meciul cerut nu există, se va afișa mesajul necriptat "Nu exista date pentru meciul cerut."
- Separat printr-un caracter spațiu, vom afișa pe aceeași linie rezultatul cerut sub forma unui întreg. (numărul minim/maxim de meciuri/lovituri)
- Atât acuratețile cât și rezultatul statisticii vor fi criptate cu cheia primită în cerință, în mod individual una de cealaltă.
- Caracterul spațiu nu va fi criptat nefăcând parte din alfabet.

Exemplul 1

Date de intrare

```

1 1
2 6 6
3 A H 1 1
4 L V 1 1
5 L H 1 1
6 L V 3 4
7 A V 2 2
8 1 1
9 2 2
10 1 2
11 2 4
12 2 2
13 -
14 OU001090.99MIN
    
```

Date de ieșire

```

1 Nava Laffey nu poate fi amplasata vertical la coordonatele (1, 1).
2 1 0 0 0 0 0
3 0 0 0 2 0 0
4 0 0 0 2 0 0
5 0 0 0 0 0 0
6 0 0 0 0 0 0
7 0 0 0 0 0 0
8
9 2 2 0 0 0 0
10 0 1 0 0 0 0
11 0 0 0 0 0 0
12 0 0 0 0 0 0
13 0 0 0 0 0 0
14 0 0 0 0 0 0
15 Jucatorul 1 a distrus o nava Laffey plasata intre coordonatele (1, 1) si (1, 2).
16 Jucatorul 2 a ratat o lovitura la coordonatele (2, 2).
17 Jucatorul 1 a ratat o lovitura la coordonatele (1, 2).
18 Jucatorul 2 a lovit o nava Laffey la coordonatele (2, 4).
19 Jucatorul 1 a distrus o nava Albacore plasata intre coordonatele (2, 2) si (2, 2).
20 Jucatorul 1 a castigat.
21 066.66.1
    
```

Explicatie

- Pe prima linie este citit caracterul "-", semnificând faptul că va fi primită o comandă necriptată, iar rezultatul acesteia va fi afișat în clar.
- Pe a doua linie este citită comanda: OU00190.99MIN, reprezentând:
 - O - Statistica se va face referitor la primul jucător
 - U - Statistica se va face referitor la un singur meci A. Deci, din cele 4 statistici posibile, se va cere ori statistica 2, ori statistica 4.
 - 001 - Meciul referitor la care ni se cere statistica.
 - 090.99 - Acuratetea X la care ne raportăm statistica, în cazul curent: 90.99%
 - MIN - Statistica ne cere numărul minim de lovituri, deci ne vom referi la statistica 4.
- Unificând informațiile extrase din comandă, ajungem la concluzia că trebuie să determinăm numărul

minim de lovituri pe care nu ar fi trebuit să le rateze jucătorul 1 în meciul 1 pentru a avea o acuratețe mai mare sau egală cu 90.99%.

- Pe prima linie din datele de ieșire, se va pune:
 - 66.66 - Reprezentând acuratețea primului jucător în primul meci. (2 lovituri nimerite din cele 3 lovituri totale)
 - .1 - Numărul minim de lovituri pe care nu ar fi trebuit să le rateze primul jucător astfel încât să atingă o acuratețe de măcar 90.99%. Pentru meciul dat, jucătorul 1 ar trebui să fi nimerit toate cele 3 lovituri pentru a atinge măcar 90.99% acuratețe, deci $3 - 2 = 1$ lovituri pe care nu ar fi trebuit să le rateze.

Algoritmul Hill Cipher

Codificarea alfabetului

Pentru acest algoritm vom asocia fiecărei litere din alfabet câte un număr de la 10 la 35, astfel încât literei A i se va asocia numărul 10, iar literei Z numărul 35. Toate cifrele de la 0 la 9 vor rămâne neschimbare în codificare. Pe deasupra, vom mai avea un caracter special "." căruia îi asociem numărul 36. În total, alfabetul nostru va avea 37 de caractere (26 de caractere alfabetice, 10 caractere numerice și un caracter special).

Criptare

Să presupunem că avem un mesaj de lungime K , pentru codificarea acestuia vom folosi o cheie de lungime K^2 . Să luăm ca exemplu mesajul "FUN" și cheia "PCLP12025". Pașii pentru criptarea mesajului sunt următorii:

- Codificarea mesajului într-un vector coloană:

$$FUN = \begin{bmatrix} 15 \\ 30 \\ 23 \end{bmatrix}$$

- Codificarea cheii într-o matrice pătratică:

$$PCLP12025 = \begin{bmatrix} 25 & 12 & 21 \\ 25 & 1 & 2 \\ 0 & 2 & 5 \end{bmatrix}$$

- Înmulțim cheia noastră la stânga mesajului original:

$$\begin{bmatrix} 25 & 12 & 21 \\ 25 & 1 & 2 \\ 0 & 2 & 5 \end{bmatrix} * \begin{bmatrix} 15 \\ 30 \\ 23 \end{bmatrix} = \begin{bmatrix} 25 * 15 + 12 * 30 + 21 * 23 & 25 * 15 + 1 * 30 + 2 * 23 & 0 * 15 + 2 * 30 + 5 * 23 \end{bmatrix}$$

$$\begin{bmatrix} 25 & 12 & 21 \\ 25 & 1 & 2 \\ 0 & 2 & 5 \end{bmatrix} * \begin{bmatrix} 15 \\ 30 \\ 23 \end{bmatrix} = \begin{bmatrix} 1218 \\ 451 \\ 175 \end{bmatrix}$$

- Aplicăm modulo lungimea alfabetului pe rezultat:

$$\begin{bmatrix} 1218 \\ 451 \\ 175 \end{bmatrix} \bmod 37 = \begin{bmatrix} 13 \\ 27 \\ 13 \end{bmatrix}$$

- Decodificăm rezultatul conform asocierii cu alfabetul:

$$\begin{bmatrix} 13 \\ 27 \\ 13 \end{bmatrix} = Y7R$$

Observație: Pentru criptarea/decriptarea unui mesaj cu o cheie de lungime K^2 și un mesaj de lungime $LEN_{MSG} > \sqrt{K^2}$, vom cripta mesajul pe secvențe de lungime $\sqrt{K^2}$, iar pentru un mesaj de lungime $LEN_{MSG} < \sqrt{K^2}$ vom lua doar primele LEN_{MSG}^2 caractere din cheie pentru a compune matricea de criptare.

Un astfel de exemplu poate fi făcut păstrând cheia "PCLP12025" și criptând mesajul "CAPYBARA". Vom cripta pe rând secvențele "CAP" și "YBA" cu cheia "PCLP12025", iar la final vom cripta secvența "RA" cu cheia "PCLP".

Decriptare

Pentru decriptare, vom presupune tot un mesaj de lungime K . Pentru decriptarea acestuia avem nevoie de aceeași cheie pe care am folosit-o la criptarea sa. Pașii algoritmului sunt următorii:

- Codificăm mesajul criptat într-un vector coloană:

$$Y7R = \begin{bmatrix} 34 \\ 7 \\ 27 \end{bmatrix}$$

- Codificăm cheia într-o matrice pătratică:

$$PCLP12025 = \begin{bmatrix} 25 & 12 & 21 \\ 25 & 1 & 2 \\ 0 & 2 & 5 \end{bmatrix}$$

- Calculăm determinantul matricei:

$$\det(PCLP12025) = \begin{vmatrix} 25 & 12 & 21 \\ 25 & 1 & 2 \\ 0 & 2 & 5 \end{vmatrix} = 25 \cdot 1 \cdot 5 + 25 \cdot 2 \cdot 21 + 12 \cdot 2 \cdot 0 - 21 \cdot 1 \cdot 0 - 25 \cdot 12 \cdot 5 - 25 \cdot 2 \cdot 2 = -425$$

- Aplicăm modulo între determinant și lungimea alfabetului:

$$-425 \mod 37 = 19$$

HINT: Verificați comportamentul operatorului modulo în C raportat la semnificația matematică din acest exemplu.

- Calculăm inversul multiplicativ al rezultatului, în raport cu lungimea alfabetului. Inversul multiplicativ al unui număr A în raport cu un număr K poate fi definit ca:

$$(A * X) \mod K = 1$$

Astfel pentru 19, inversul multiplicativ în raport cu 37 este:

$$(19 * X) \mod 37 = 1$$

$$X = 2$$

- Calculăm adjuncta cheii și aplicăm modulo lungimea alfabetului pe aceasta:

$$PCLP12025^* = \begin{bmatrix} 1 & -18 & 3 \\ -125 & 125 & 475 \\ 50 & -50 & -275 \end{bmatrix} \mod 37 = \begin{bmatrix} 1 & 19 & 3 \\ 23 & 14 & 31 \\ 13 & 24 & 21 \end{bmatrix}$$

- Înmulțim rezultatul de la pasul anterior cu matricea adjunctă a cheii și aplicăm din nou modulo 37:

$$PCLP12025^{-1} = 2 * \begin{bmatrix} 1 & 19 & 3 \\ 23 & 14 & 31 \\ 13 & 24 & 21 \end{bmatrix} \mod 37 = \begin{bmatrix} 2 & 1 & 6 \\ 9 & 28 & 25 \\ 26 & 11 & 5 \end{bmatrix}$$

- Înmulțim cheia la dreapta mesajului criptat și aplicăm din nou modulo:

$$\begin{bmatrix} 2 & 1 & 6 \\ 9 & 28 & 25 \\ 26 & 11 & 5 \end{bmatrix} * \begin{bmatrix} 34 \\ 7 \\ 27 \end{bmatrix} \mod 37 = \begin{bmatrix} 237 \\ 1177 \\ 1096 \end{bmatrix} \mod 37 = \begin{bmatrix} 15 \\ 30 \\ 23 \end{bmatrix}$$

- Decodificăm matricea coloană rezultată în string-ul asociat alfabetului:

$$\begin{bmatrix} 15 \\ 30 \\ 23 \end{bmatrix} = FUN$$

Exemplul 1

Date de intrare

```

1 1
2 6 6
3 A H 1 1
4 L V 1 1
5 L H 1 1
6 L V 3 4
7 A V 2 2
8 1 1
9 2 2
10 1 2
11 2 4
12 2 2
13 .EXAMPLE.
14 QC.EMEWS5TOY8A
    
```

Date de ieșire

```

1 Nava Laffey nu poate fi amplasata vertical la coordonatele (1, 1).
2 1 0 0 0 0 0
3 0 0 0 2 0 0
4 0 0 0 2 0 0
5 0 0 0 0 0 0
6 0 0 0 0 0 0
7 0 0 0 0 0 0
8
9 2 2 0 0 0 0
10 0 1 0 0 0 0
11 0 0 0 0 0 0
12 0 0 0 0 0 0
13 0 0 0 0 0 0
14 0 0 0 0 0 0
15 Jucatorul 1 a distrus o nava Laffey plasata intre coordonatele (1, 1) si (1, 2).
16 Jucatorul 2 a ratat o lovitura la coordonatele (2, 2).
17 Jucatorul 1 a ratat o lovitura la coordonatele (1, 2).
18 Jucatorul 2 a lovit o nava Laffey la coordonatele (2, 4).
19 Jucatorul 1 a distrus o nava Albacore plasata intre coordonatele (2, 2) si (2, 2).
20 Jucatorul 1 a castigat.
21 NN4ODKFE
    
```

Explicatie

- Datele de intrare vor fi aceleași ca în exemplul 1 pentru task 2.
- Adicional, este dată o cheie de criptare ".EXAMPLE.", ce va fi codificată și salvată sub forma unei matrici:

$$\begin{bmatrix} 36 & 14 & 33 \\ 10 & 22 & 25 \\ 21 & 14 & 36 \end{bmatrix}$$

- Statistica cerută - care este aceeași ca în exemplul 1 pentru task 2 - este criptată în input cu cheia menționată anterior.

- Pentru a determina statistica din cerință, vom decripta pe blocuri linia "QC.EMEWS5TOY8A". Blocurile pe care le decodificăm sunt următoarele:

1. "QC."
2. "EME"
3. "WS5"
4. "TOY"
5. "8A"

- Pentru ultimul bloc ("8A") vom decodifica folosind doar primele $2^2 = 4$ valori din cheie:

$$\begin{bmatrix} 36 & 14 \\ 33 & 10 \end{bmatrix}$$

- Pentru decriptare urmăm pașii algoritmului de la pagina anterioară.
- După determinarea rezultatului statisticii, vom cripta rezultatul - 066.66.1 - tot pe blocuri, cu matricea cheie. Blocurile de criptat sunt următoarele:
 1. "066"
 2. ".66"
 3. ".1"
- La fel ca la decriptare, ultimul bloc va fi criptat folosind cheia redusă cu doar 4 elemente.
- Pentru criptare urmăm pașii algoritmului de la pagina anterioară.

Regulament

Regulamentul general al temelor se găsește pe ocw (**Temele de casă**). Vă rugăm să îl citiți integral înainte de a continua cu regulile specifice acestei teme.

Arhivă

Soluția temei se va trimite ca o arhivă **zip**. Numele arhivei trebuie să fie de forma **Grupă_NumePrenume_TemaX.zip** - exemplu: 315CA_BirladeanuRaluca_Tema2.zip.

Arhiva trebuie să conțină în directorul **RĂDĂCINĂ** ****doar**** următoarele:

- Codul sursă al programului vostru (fișierele **.c** și eventual **.h**).
- Un fișier **Makefile** care să conțină regulile **build** și **clean**.
 - Regula **build** va compila codul vostru și va genera ****doar**** un singur executabil numit **battleships**
 - Regula **clean** va șterge **toate** fișierele generate la build (executabile, binare intermediare etc).
- Un fișier **README** care să conțină prezentarea implementării alese de voi. **NU** copiați bucăți din enunț.

Arhiva temei **NU** va conține: fișiere binare, fișiere de intrare/ieșire folosite de checker, checkerul, orice alt fișier care nu este cerut mai sus.

Numele și extensiile fișierelor trimise **NU** trebuie să conțină spații sau majuscule, cu excepția fișierului **README** (care are numele scris cu majuscule și nu are extensie).

Nerespectarea oricărei reguli din secțiunea **Arhivă** aduce un punctaj **NUL** (0) pe temă.

Checker

Pentru corectarea acestei teme vom folosi scriptul **check** din arhiva **check_capybara_remastered.zip** din secțiunea de resurse asociată temei. Vă rugăm să citiți **README-checker.md** pentru a ști cum să instalați și utilizați checker-ul.

Punctaj

Distribuirea punctajului:

- Implementarea jocului battleships: 20p
- Implementarea statisticilor: 30p
- Implementarea criptării și a decriptării: 30p
- Claritatea și calitatea codului (inclusiv modularizare): 10p
- Claritatea explicațiilor din **README**: 10p

ATENȚIE! Punctajul maxim pe temă este **100p**. Acesta reprezintă 0.75p din nota finală la această materie. La această temă nu se vor acorda punctaje bonus. **O temă care nu folosește alocare dinamică NU va fi punctată.**

Reguli și precizări

- Punctajul pe teste este cel acordat de script-ul **check**, rulat pe **Moodle**. Echipa de corectare își rezervă dreptul de a depuncta pentru orice încercare de a trece testele fraudulos (de exemplu prin hardcodare, etc).

- Punctajul pe calitatea explicațiilor și a codului se acordă în mai multe etape:
 - **corectare automată**
 - * Checker-ul va încerca să detecteze în mod automat probleme legate de coding style și alte aspecte de organizare a codului.
 - * Acesta va puncta cu maxim 20p dacă nu sunt probleme detectate în mod automat.
 - * Punctajul se va acorda proporțional cu numărul de puncte acumulate pe teste din cele 80p.
 - * Checker-ul poate să aplice însă și penalizări (exemplu pentru warninguri la compilare) sau alte probleme descoperite la runtime.
 - **corectare manuală**
 - * Tema va fi corectată manual și se vor verifica și alte aspecte pe care checker-ul nu le poate prinde. Recomandăm să parcurgeți cu atenție tutorialul de **coding-style** de pe ocw.cs.pub.ro.
 - * Codul sursă trebuie să fie însoțit de un fișier README care trebuie să conțină informațiile utile pentru înțelegerea funcționalității, modului de implementare și utilizare a programului. Acesta evaluează, de asemenea, abilitatea voastră de a documenta complet și concis programele pe care le produceți și va fi evaluat, în mod analog coding-style-ului, de către echipa de asistenți. În funcție de calitatea documentației, se vor aplica depunctări sau bonusuri.
 - * La corectarea manuală se va acorda un bonus de maximum 10 puncte pentru modularizare. Deprinderea de a scrie cod sursă de calitate, este un obiectiv important al materiei. Sursele greu de înțeles, modularizate neadecvat sau care prezintă hardcodări care pot afecta semnificativ mentenabilitatea programului cerut, pot fi depunctate adițional. Penalizarea pentru modularizare defectuoasă sau absentă complet, poate să fie oricât de mare.
 - * În această etapă se pot aplica depunctări oricât de mari (chiar și totale). Orice rezultat / punctaj acordat automat de checker, poate fi anulat sau suprascris de către echipa de PCLP la corectarea manuală.
 - * O temă care **NU** compilează cu -Wall -Wextra este depunctată la corectarea manuală cu 5p (punctajul echivalent pentru warnings). Echipa de PCLP vă recomandă să compilați cu -Wall -Wextra -Werror.
 - * **Tema trebuie să reprezinte exclusiv munca studentului!** Echipa va aplica regulamentele în vigoare pentru situațiile de fraudă / plagiat, pentru orice nerespectare a acestor reguli, incluzând, dar nelimitându-se la: copierea de la colegi (se aplică pentru ambele persoane implicate) sau din alte surse a unor părți din temă, prezentarea ca rezultat personal a oricărei bucăți de cod provenită din alte surse necitate sau neaprobate în prealabil (site-uri web, alte persoane, cod generat folosit AI / LLM-uri etc.).

Alte precizări

- Implementarea se va face în limbajul **C**, iar tema va fi compilată și testată **DOAR** într-un mediu **LINUX**. Nerespectarea acestor reguli aduce un punctaj **NUL**.
- Tema trebuie trimisă sub forma unei arhive pe site-ul cursului **curs.upb.ro**.
- Tema poate fi trimisă de oricâte ori fără depunctări până la deadline. Mai multe detalii se găsesc în regulamentul de pe [ocw](http://ocw.cs.pub.ro).
- O temă care **NU** compilează **NU** va fi punctată.
- O temă care compilează, dar care **NU** trece niciun test **NU** va fi punctată.
- Punctajul pe teste este cel acordat de **check** rulat pe **platforma remote a echipei de PCLP**. Echipa de corectare își rezervă dreptul de a depuncta pentru orice încercare de a trece testele fraudulos (de exemplu prin hardcodare).
- Ultima temă submisă pe Moodle poate fi rulată de către responsabili / echipa de PCLP de mai multe ori în vederea verificării faptului că nu aveți bug-uri în sursă. În cazul obținerii punctajelor diferite la rulări multiple, vom păstra minimul dintre punctaje. Vă recomandăm să verificați local tema de mai multe ori pentru a verifica că punctajul este mereu același, apoi să încărcați tema. De asemenea, verificați că punctajul de pe platformă este cel așteptat - singurul punctaj / feedback relevant este cel de pe platforma remote PCLP (**vmchecker-next**).